

Indexing and storage

This briefing is the *write path*: how a PDF or Word file becomes rows and indexes in Postgres. Briefing 4 is the *read path*: how `search_documents` queries those same rows. There is no separate search cluster. Chat stays the control plane.

Question	What we actually have
Hosted vector DB (Pinecone, Weaviate, Elastic kNN)?	No. A vector(768) column on <code>document_chunks</code> in the same Postgres (Neon or Docker). Index: HNSW, cosine ops. Query uses <code>embedding <=> query::vector</code> .
BM25?	No. English Postgres full-text search: GIN on <code>to_tsvector('english', contextual_text)</code> , query via <code>websearch_to_tsquery</code> , rank via <code>ts_rank_cd</code> . BM25 as a third arm is deferred (<code>docs/retrieval.md</code>).
Elasticsearch / standalone search product?	No. Do not replace agentic grep with one. Hybrid retrieval already runs under chat.
Is the report body indexed?	No. TipTap JSON lives in <code>report_sections</code> . Chat uses <code>read_section</code> . Only attachment pages are chunked.

Two stores, one pipeline. Object storage holds *bytes* (preview, download, re-ingest). Postgres holds *searchable text + vectors*. Search never greps GCS. If ingest did not finish, there is nothing to retrieve.

1. Overall: write path then read path

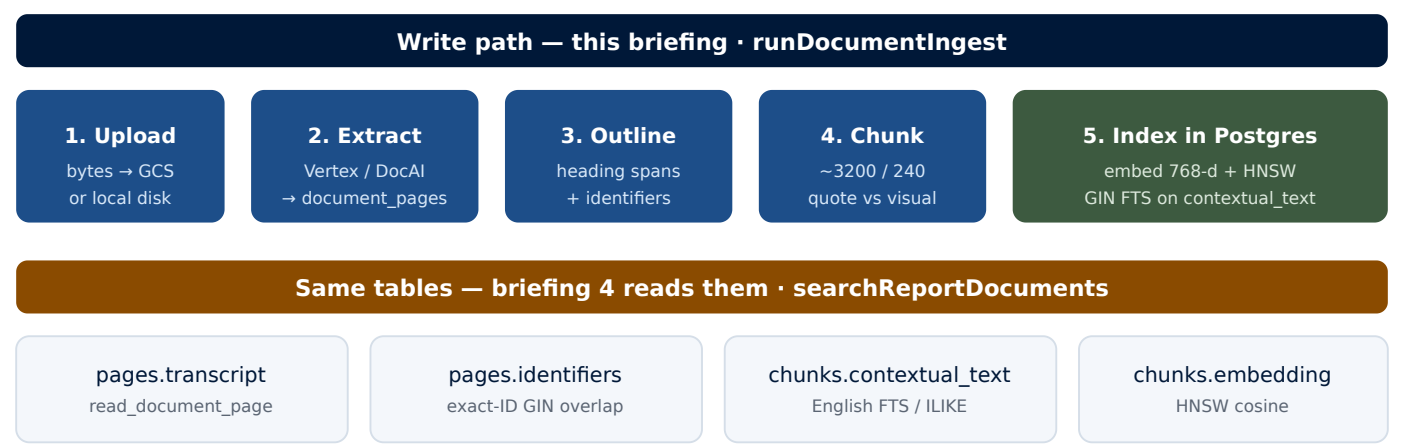


Figure 1. Indexing is ingest writing Postgres. Retrieval is SQL against those rows. Do not invent a third system in between.

Join is always the report's live ingest: `report_attachments.active_ingest_run_id = document_chunks.ingest_run_id`. Superseded runs are invisible to search. Library links join by `asset_id` when set; `eval / legacy` rows with null `asset_id` join on `attachment_id`. Citations stay filename + page — chunk ids change on re-ingest. Living plans: `docs/document-ingest-pipeline.md` (write) · `docs/retrieval.md` (read). Code: `run-document-ingest.ts` · `retrieval.ts`.

2. How input documents are processed

Entry: startDocumentIngest → runDocumentIngest (src/lib/attachments/run-document-ingest.ts). Extract + embed are **Vertex-only** (GOOGLE_VERTEX_PROJECT). A Gateway key is not enough. Stub: ALLOW_TEST_STUB_DOCUMENT_INGEST.

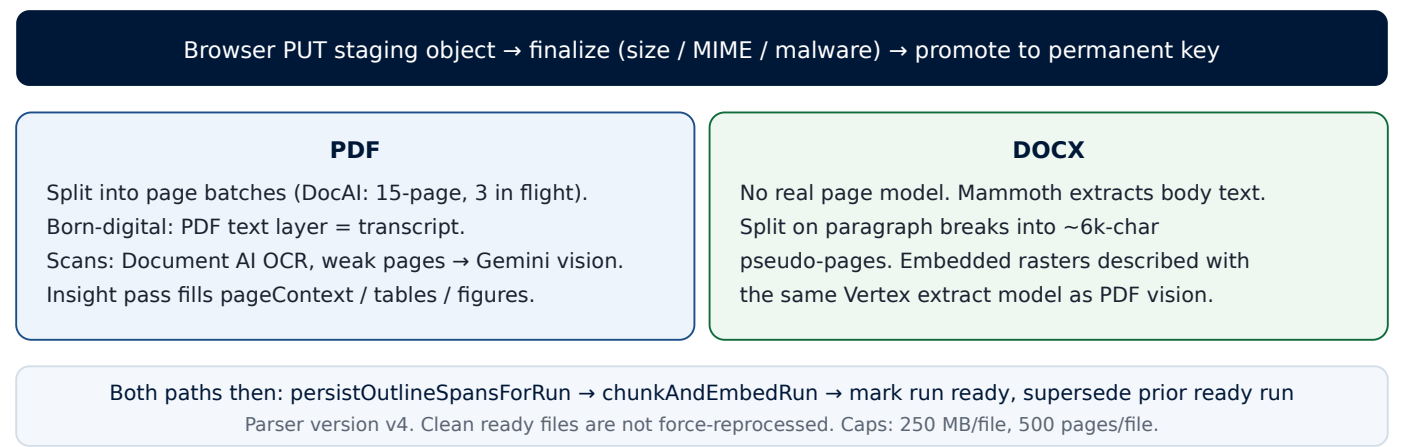


Figure 2. Extract model is Gemini Flash-Lite at Vertex global. OCR is a regional Document AI processor. Embeddings stay on GOOGLE_VERTEX_LOCATION (often us-central1). Do not conflate those locations.

Where bytes live

Layer	Table / backend	What it stores
Canonical bytes	attachment_assets + GCS (GCS_BUCKET) or local disk	The PDF/DOCX. Vault at /vault. Linking into a report does not copy bytes or count storage twice.
Report link	report_attachments	Filename, folder, active_ingest_run_id, processing status. Soft-delete via deletedAt.
One extract attempt	attachment_ingest_runs	document_summary, outline spans pointer, status pending → running → ready failed cancelled.
PDF slices	document_ingest_batches	Temp batch objects while extract is in flight.

Production: ATTACHMENT_STORAGE_BACKEND default GCS + WIF. Local uploads need ATTACHMENT_STORAGE_BACKEND=local **and** ALLOW_LOCAL_ATTACHMENT_STORAGE=true.

3. What Postgres stores after ingest

Searchable evidence is three grains. Metadata is filterable columns (file / page / span), not extra prose stuffed into every chunk. Chunks inherit page columns via `pageId`.

document_pages (one row per PDF page or DOCX pseudo-page)

`transcript · visual_interpretation · page_context · outline_title · identifiers[] · has_table / has_figure (nullable)`

GIN on identifiers. `Chat read_document_page` returns the transcript (capped 12k). This is the quote ledger’s source of truth.

document_outline_spans (heading range + unioned identifiers)

Written after pages exist, before chunk/embed. `document_outline` prefers stored spans; else live `buildOutlineFromStoredPages`.

document_chunks (the search index)

`raw_text · contextual_text = “Document: file | Page N | pageContext” + raw`

`source_kind quote | visual_interpretation · embedding vector(768) · HNSW cosine`

GIN FTS on `to_tsvector('english', contextual_text)` — expression must match the query byte-for-byte

Unique (`ingest_run_id`, `page_number`, `ordinal`). ~3200 chars, 240 overlap, split on paragraph/sentence.

Figure 3. `list_attachments` is not this index. It walks the Attachments tree (count / folders / types / status / filename). Which files mention a fact *inside* a PDF is still chunk search.

How a page becomes chunks

`chunkDocumentPages` in `chunk-pages.ts`. Transcript and visual interpretation are chunked as separate sources so a figure caption does not drown the quote, and ranking can prefer quote over `visual_interpretation`.

Constant	Value
DEFAULT_CHUNK_MAX_CHARS	3_200
DEFAULT_CHUNK_OVERLAP_CHARS	240
Embedding model	gemini-embedding-001, task RETRIEVAL_DOCUMENT, 768-d, batches of 32
Query embedding (briefing 4)	Same model, task RETRIEVAL_QUERY
HNSW index	<code>document_chunks_embedding_hnsw_idx vector_cosine_ops</code>
FTS index	<code>document_chunks_contextual_text_fts_en_idx GIN to_tsvector('english', contextual_text)</code>

There is no BM25 index to look at. Keyword search is Postgres `tsvector` / `tsquery` with English stemming. That is closer to a Lucene TF-IDF variant than Okapi BM25. Adding BM25 would be a new arm fused with vector + FTS — explicitly deferred, not missing by accident.

4. Status until search can see the file

Attachment: uploading → validating → queued → processing → ready (or failed). Ingest run: pending → running → ready | failed | cancelled. A new successful run marks the previous ready run superseded. Chat lists ready docs in the context map (filename + sanitized documentSummary). Untrusted extract text is sanitized in chat/tools, not in the attachments package.

Andrei · indexing and storage · briefing 4 queries these tables · code is source of truth